

CogStack

Persistent AI Cognition and Multi-Vendor Cognitive Stacking

Niels Goldstein

Defensive Publication · June 3, 2026 · MIT License · github.com/nielsg2/cogstack-spec

Scope: This disclosure concerns only the persistent-cognition and multi-agent orchestration architecture described herein, and should not be construed as describing, disclosing, or relating to any other system or subject matter.

Abstract

CogStack is a two-layer architecture for autonomous AI agent systems. The first layer, the *persistent cognition substrate*, externalizes the context produced by interactive AI sessions, compresses it into a durable memory store, and rehydrates it into subsequent stateless API calls so agents reason continuously across sessions and environments. The second layer, *multi-vendor cognitive stacking*, orchestrates agents from different model providers and families so the composite system exhibits properties that no single constituent produces alone — specifically, decorrelated diversity in which each agent's failure modes do not coincide with the others'. CogStack is designed as the orchestration complement to AIDL (AI Directive Language), which provides the governance grammar that bounds agent behavior and weights adjudication by provider independence. Together they form a matched two-layer system: CogStack orchestrates decorrelated cognition; AIDL governs and adjudicates it. This disclosure is published as prior art under the MIT License.

Background and Problem

Statefulness. Generative AI systems are fundamentally stateless. Interactive sessions produce rich context — decisions, rationale, queued tasks, accumulated state — that evaporates when the session ends. API-driven automation is disconnected from this high-context work. The result: agents cannot reason over time, cannot resume interrupted work, and cannot operate headlessly without a human re-establishing context at each invocation.

Redundancy without diversity. A common attempt at multi-agent reliability is to run the same prompt through multiple instances of the same model and vote. This achieves variance reduction but not genuine decorrelation: agents built on the same base model share architecture, training corpus, post-training shaping, and failure modes. Their agreement is correlated rather than corroborating. A system that cannot distinguish correlated agreement from genuine independent consensus manufactures false confidence. CogStack addresses both deficiencies with two cooperating architectural layers.

Layer 1 — Persistent Cognition Substrate

Context Compression Engine

Receives the artifact of an interactive AI session and transforms it into compact **semantic blocks**: typed, tagged, machine-readable representations of essential intent, task state, and contextual constraints, sized to fit within a future API prompt. Block types: task, decision, constraint, background, open-question. Each block carries confidence, recency, and relevance signals governing trimming at rehydration time. Compression is intentionally lossy; the target is operational continuity, not verbatim recovery.

Memory Store

A durable repository of semantic blocks, execution records, and directive state, persisting across process restarts and environment changes. Format-agnostic: structured log files, graph models, or vector indices for embedding-based retrieval are all valid substrates. Durability is the defining constraint.

Rehydration Pipeline

At execution time, constructs an enriched API prompt by: (1) reading the next directive from the queue; (2) querying the Memory Store for contextually relevant semantic blocks; (3) ranking and trimming blocks to fit the target model's context budget; (4) composing the prompt from system instructions, rehydrated context, and current task directive. Context budget is a hard constraint; the pipeline never exceeds the model's context window.

Execution Daemon (gpt-cogd)

A headless background process that polls the Directive Manager for executable tasks, invokes the Rehydration Pipeline, calls the target AI API, records responses in the Memory Store, updates task state, and appends every action to the Audit Log. Designed for long-running, unattended operation without a GUI.

Directive Manager

A task queue and scheduler where each directive specifies: description, preconditions (evaluated against the Memory Store for data-dependent ordering), priority, execution constraints, and status (pending, ready, in-progress, completed, blocked, deferred). Enables conditional execution ordering without a separate workflow engine.

Audit Log

An append-only, versioned record of every execution event. Supports governance review, debugging, reputation scoring in AIDL-governed deployments, and rollback of bad executions.

Plugin / Extension Bus

A modular extension mechanism by which new capabilities — tool integrations, alternative memory backends, custom compression strategies, additional provider adapters — are added without modifying the core, subject to the same audit and write-scope discipline as the core.

Layer 2 — Multi-Vendor Cognitive Stacking

Decorrelated Diversity

The genuine gain from multi-agent systems requires agents whose training corpora, architectures, and post-training shaping differ enough that their failure modes do not coincide. Nominal vendor diversity (different logos, shared base models) does not achieve this. A CogStack deployment selects agents with the explicit intent to maximize genuine independence, not nominal headcount.

Composition Topologies

- **Parallel-and-adjudicate.** All agents receive the same task independently and produce outputs simultaneously; an adjudication step (governed by AIDL's reputation-weighted voting) resolves disagreements. Suited to tasks where correctness can be compared across outputs.
- **Sequential / pipeline.** Agents specialize by stage — draft, critique, synthesize — with each stage's output feeding the next. Suited to tasks where critique benefits from seeing a first attempt.
- **Adversarial.** One or more agents are explicitly tasked to find flaws in the primary output; disagreement is the designed product. Suited to high-stakes outputs where sycophantic agreement is a risk.

- **Router / dispatch.** Task type determines agent assignment based on a profile of each model's known strengths and failure modes. Suited to heterogeneous workloads where no single model dominates across task categories. Topologies are composable.

Invocation Criteria

Cognitive stacking multiplies API cost and latency; it is not the default execution path. CogStack specifies invocation criteria: high-stakes outputs, correctness-critical tasks, high-ambiguity inputs, and anti-sycophancy requirements. A conforming deployment names the conditions under which stacking is triggered.

Error Modes and Governance Dependency

Stacking can amplify rather than cancel error when a majority of agents share a structural bias (*correlated consensus laundering*) or when the adjudication mechanism is itself biased (*adjudication layer bias*). These failure modes are why CogStack is designed as the orchestration complement to AIDL's governance layer: AIDL's provider-independence discounting and reputation weighting are the architectural guardrails that prevent the stack from laundering correlated agreement into false confidence.

AIDL Integration

Layer	System	Function
Orchestration	CogStack	Assembles agents, manages persistent memory, executes directives
Governance	AIDL	Defines roles, accumulates reputation, adjudicates conflicts

CogStack dispatches tasks and manages context; AIDL governs how agent authority is assigned, how disputes are resolved, and how reputation evolves from observed behavior. The layers are independently deployable but designed for joint use. A deployment without governance has no principled mechanism for weighting disagreements; a deployment without persistent memory has no continuity across sessions.

Disclosed Future Embodiments

The following embodiments are disclosed, both to document design intent and to broaden the prior-art footprint of this disclosure:

- **Dual-mode (voice + text) interaction** — text-to-speech output with a synchronized live transcript, driving the same persistent-cognition loop by voice or text interchangeably.
- **API chaining and orchestration** — composing multiple external tools and services into a single directive's execution, with the Memory Store carrying intermediate state across the chain.
- **Autonomous agent / task queue with memory injection** — long-running autonomous tasks that carry persistent context forward across invocations rather than starting cold.
- **Secure execution sandbox** — a constrained execution boundary (sandboxed VM or equivalent) for AI-directed automation, containing and making reversible the actions taken by the daemon.
- **Cross-vendor consensus and contradiction detection** — routing across multiple providers with explicit detection of agreement and disagreement between models (the operational core of cognitive stacking), with embedding-indexed memory of recurring patterns and operator preferences, using local inference where appropriate and cloud inference where necessary.

Associated Terminology

The following terms are disclosed as descriptors of the methodology embodied by CogStack, to place them in the attributed public record:

- **Cognitive stacking** — composing agents from genuinely independent providers so the composite exhibits decorrelated diversity that no single constituent provides.
- **Semantic block** — a typed, tagged, compressed unit of session context sized for re-injection into a future prompt.
- **Rehydration** — the explicit reconstruction of an agent's working context from durable memory at execution start, rather than relying on a vendor's opaque native memory.
- **Cognitive instruction set** — a human-readable, machine-actionable directive package issued to an agent or daemon, functioning as an instruction set for autonomous execution.
- **Guard-railed task injection** — delivery of an isolated task payload with enforced constraints on output format, execution boundary, and error handling.

Provenance and Open-Publication Statement

CogStack's persistent-AI-cognition substrate — a structured AI-interaction history archive and AI session continuity — first appears under version control on June 8, 2025, the same day as the companion AIDL directive framework; session-writer/closer scaffolding, a workflow-continuity document, and a task queue follow on June 9; the system was named “CogStack” and given a dedicated repository on June 19, 2025, and integrated as the cognition module of a persistent daemon by June 23, 2025 (AIDL's language was named June 25, 2025). The two were developed as cooperating layers of the same architectural system. This repository is a curated public extract of that private working set, published as a contribution and as prior art. No patent rights are asserted. The work is released under the MIT License so the underlying ideas remain free for anyone to build on.

License

Copyright (c) 2025 Niels Goldstein. Released under the MIT License. Permission is granted, free of charge, to any person obtaining a copy of this work, to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies, subject to inclusion of the copyright and permission notice. The work is provided “as is”, without warranty of any kind.